Dharamdutt Kumar

dharam53579@gmail.com | +91 9631950092 | Lucknow, India (Remote)

Automation & QA Developer Assessment

Task 1 — Web App QA & Debug Report

Target Application: Conduit (RealWorld Demo App)

URL Evaluated: `https://demo.realworld.io`

Testing Environment: Chrome 120+, Network Throttling, LocalStorage Inspection

Primary Flows Tested: Sign-up, Auth, Feed, CRUD Articles, Comments, Settings

Total Issues Documented: 6 Production Deficiencies

Severity Distribution: 1 Critical • 2 High • 2 Medium • 1 Low

1. DETAILED DEFECT & BUG TRACKING MATRIX

#	TITLE / SUMMARY	STEPS TO REPRODUCE	EXPECTED VS ACTUAL	SEVERITY	SUSPECTED CAUSE
1	**Stored Cross-Site Scripting (XSS) via Unsanitized Markdown**	1. Authenticate and click **New Article**. 2. In article markdown body, inject: `<img src="x" onerror="alert(document.domain)">`. 3. Click **Publish Article**. 4. View published article.	**Expected:** Raw HTML tags and executable attributes (`onerror`, `onclick`) should be sanitized by DOMPurify before rendering. **Actual:** Injected HTML executes in viewer's browser context, triggering JavaScript alert.	CRITICAL	Markdown parser renders directly into `dangerouslySetInnerHTML` without HTML sanitization.
2	**Form Double-Submission & Slug Collision under Network Latency**	1. Fill out **New Article** form. 2. Throttle network to **Slow 3G** in DevTools. 3. Click **Publish Article** twice rapidly before redirect.	**Expected:** Submit button disables and displays loading spinner immediately on first click. **Actual:** Two concurrent POST requests fire; second fails with `422 Unprocessable Entity (slug must be unique)`.	HIGH	Missing `isSubmitting` state disabling the button; missing request debounce/race-condition guard.
3	**Expired JWT Token Causes Silent Infinite Loading State**	1. Log in to establish valid session. 2. In DevTools > Storage, corrupt or truncate the `jwtToken` value. 3. Refresh page or click **Your Feed**.	**Expected:** 401 response should trigger token wipe, auth state reset, and redirect to `/login`. **Actual:** Feed spinner runs indefinitely; UI displays user as logged-in with empty white screen.	HIGH	Missing HTTP client interceptor to capture 401 errors, clear `localStorage`, and redirect.
4	**Layout Shift & Broken Image on Unreachable Avatar URL**	1. Go to **Settings**. 2. Set Profile Picture URL to an invalid/404 URL (e.g. `https://broken-img.xyz/404.png`). 3. Save and view authored posts or navbar.	**Expected:** Fallback to default SVG placeholder on image load failure; fixed dimensions set. **Actual:** Broken image placeholder renders, distorting navbar layout and article header alignment.	MEDIUM	Missing `onError` fallback handler on `<img>` element and absence of URL regex validation.
5	**Pagination Offset State Desynchronization Across Feed Tabs**	1. In **Global Feed**, navigate to page 3 (offset = 20). 2. Click **Your Feed** tab (where total user articles < 5).	**Expected:** Switching feed tabs should reset active pagination offset to page 1. **Actual:** App requests `GET /api/articles/feed?offset=20`, rendering false empty state.	MEDIUM	Feed selection action updates feed type in state without dispatching a pagination reset action.
6	**Unvalidated Duplicate & Whitespace Tag Badges in Editor**	1. In article editor, enter tag `"qa"`, press Enter. 2. Type `"qa"` again or enter whitespace `" "`.	**Expected:** Whitespace trimmed, duplicate tags discarded via `Set` or `includes()` check. **Actual:** Duplicate and blank tag badges accumulate in UI; deleting one removes duplicates simultaneously.	LOW	Tags appended to array without `trim()` validation and uniqueness verification.

Dharamdutt Kumar
dharam53579@gmail.com | +91 9631950092 | Task 1 — Root Cause Analysis

2. IN-DEPTH ROOT CAUSE ANALYSIS (ISSUE #1: STORED XSS VIA MARKDOWN)

Root Cause Investigation & Architecture Remediation

What is happening: When an author creates or updates an article containing raw HTML markup with embedded JavaScript event handlers (such as `<img src=x onerror=alert(document.cookie)>`), the markdown parser parses this markup verbatim. When any visitor navigates to the article page, the injected payload executes immediately inside their authenticated browser session, granting attackers access to session storage, JWT auth tokens, and full account impersonation capabilities.

Why it happens (The "Vibe-Coding" Trap): In AI-assisted or rapid prototype builds, developers frequently use standard markdown compilation packages (such as `marked`) and directly inject the resulting HTML string into the React/Vue component tree using `dangerouslySetInnerHTML={{ __html: compiledMarkdown }}`. The developer assumed that the markdown library intrinsically sanitizes unsafe HTML tags. However, the CommonMark and GitHub Flavored Markdown specifications permit raw HTML passthrough by default unless an explicit DOM sanitizer is integrated.

How to Fix (Engineering Solution):
1. **Client-Side Sanitization:** Introduce `DOMPurify` to sanitize the compiled markdown output before injecting it into the DOM.
2. **Backend Defense-in-Depth:** Implement server-side input validation on the `/api/articles` controller using an HTML sanitizer before persisting article records to the database.
3. **Content Security Policy (CSP):** Deploy strict HTTP response headers (`Content-Security-Policy: default-src 'self'; script-src 'self';`) to block inline script execution as an architectural backstop.

```
// Remediated Markdown Component Implementation (React / Vue)
import DOMPurify from 'dompurify';
import { marked } from 'marked';

export const SafeArticleBody = ({ rawMarkdown }) => {
  // Step 1: Parse markdown to HTML
  const rawHtml = marked.parse(rawMarkdown || '');

  // Step 2: Strip all unauthorized tags, scripts, and executable event handlers
  const sanitizedHtml = DOMPurify.sanitize(rawHtml, {
    ALLOWED_TAGS: ['p', 'b', 'i', 'em', 'strong', 'a', 'h1', 'h2', 'h3', 'code', 'pre', 'ul', 'ol', 'li',
'blockquote'],
    ALLOWED_ATTR: ['href', 'target', 'rel']
  });

  return <div className="article-content" dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />;
};
```

QA Takeaway for AI-Built ("Vibe-Coded") Applications:

AI code generators frequently produce functional UI logic while omitting defensive programming guards—such as client-side sanitization, network debouncing, token expiration interceptors, and error boundaries. Establishing automated regression suites and static analysis in the CI/CD pipeline prevents these latent vulnerabilities from escalating into production outages.